MACHINE LEARNING-GUIDED CONFLICT PRIORITIZATION FOR HYBRID CBS-LACAM PATH PLANNING

A SUMMER INTERNSHIP REPORT

Submitted by

VARSHA T - 810023205102



DEPARTMENT OF INFORMATION TECHNOLOGY

ANNA UNIVERSITY, TIRUCHIRAPPALLI – 620 024

UNIVERSITY COLLEGE OF ENGINEERING (BITCAMPUS), ANNA UNIVERSITY, TIRUCHIRAPALLI – 620024.

DEPARTMENT OF INFORMATION TECHNOLOGY

BONAFIDE CERTIFICATE

This is to certify this summer internship project report **"Machine learning-guided conflict prioritization for hybrid CBS-LaCAM path planning"** is the Bonafide work of **VARSHA T (810023205102)** has satisfactory completed the course of **CS3711 – SUMMER INTERNSHIP** for **SEVENTH** semester, who carried out the internship **B.TECH INFORMATION TECHNOLOGY** during the academic year 2026-2027.

Faculty in charge **Head of the department**

Submitted for the Internship Viva-Voce examination held on _____

ABSTRACT

Multi-Agent Path Finding (MAPF) is a challenging problem in which multiple agents must move from their respective start locations to assigned goal locations without colliding with one another. This problem is widely encountered in warehouse automation, mobile robotics, autonomous transportation, and other multi-agent systems where safe and efficient path planning is essential. As the number of agents increases, the complexity of finding collision-free paths also increases, making it necessary to develop efficient planning algorithms.

The objective of this internship was to design and implement a Hybrid Multi-Agent Path Finding algorithm by combining LaCAM, Conflict-Based Search (CBS), and Machine Learning-based conflict prioritization. LaCAM was used to generate initial paths quickly, while CBS was employed to resolve conflicts and ensure that the final solution remained collision-free. To improve the conflict resolution process, a Random Forest regression model was introduced to predict the priority of detected conflicts based on features such as conflict type, conflict time, nearby obstacles, nearby agents, make span, and sum of costs. Instead of resolving conflicts in a fixed order, the proposed system identifies the most significant conflict and resolves it first, thereby improving the efficiency of the search process.

The algorithm was developed using Python and Pygame with an interactive interface that allows users to create grid environments, place obstacles and agents, execute different planning algorithms, and observe the generated paths. Additional features such as automatic detection of unreachable agents, CBS fallback during stagnation, and performance analysis were incorporated to improve the reliability of the system.

The proposed hybrid approach was evaluated on multiple grid environments with different numbers of agents and obstacle densities. Experimental results showed that the Random Forest model effectively prioritized conflicts, enabling the hybrid solver to reduce unnecessary conflict resolution in many scenarios while maintaining collision-free solutions. When complex situations caused the learning-guided strategy to stagnate, the automatic CBS fallback successfully completed the search, ensuring reliable performance. The results demonstrate that integrating machine learning with classical MAPF algorithms can improve conflict selection while preserving the correctness and robustness of the overall planning process.

TABLE OF CONTENT

CHAPTER	CONTENT	PAGE NO.
1	**INTRODUCTION AND OBJECTIVES**	
	1.1. INTRODUCTION	1
	1.2. OBJECTIVES OF THE INTERNSHIP	2
	1.3. SCOPE OF THE INTERNSHIP	3
2	**INSTITUTE PROFILE**	4
3	**DAILY WORK REPORT**	6
4	**LITERATURE REVIEW / SYSTEM STUDY**	
	4.1 LITERATURE REVIEW	9
	4.1.1 CONFLICT BASED SEARCH(CBS)	9
	4.1.2 LAZY CONSTRAINTS ADDITION SEARCH	9
	4.1.3 ML – BASED CONFLICT	10
	4.2 PROBLEM STATEMENT	10
	4.3 PROPOSED METHODOLOGY	11
5	**SYSTEM REQUIREMENTS & ARCHITECTURE**	
	5.1 SYSTEM ARCHITECTURE	15
	5.2 MODULE DESCRIPTION	17
	5.2.1 GRID MANAGEMENT MODULE	17
	5.2.2 LaCAM PATH PLANNING MODULE	17
	5.2.3 CONFLICT DETECTION MODULE	17
	5.2.4 FEATURE EXTRACTION MODULE	17

	5.2.5 MACHINE LEARNING MODULE	17
	5.2.6 CBS CONFLICT RESOLUTION MODULE	17
	5.2.7 CBS CONFLICT RESOLUTION MODULE	18
	5.3 MACHINE LEARNING MODEL	18
	5.4 CONFLICT FEATURE EXTRACTION	19
6	**DEVELOPMENT AND RESULT ANALYSIS**	
	6.1 SYSTEM IMPLEMENTATION	20
	6.2 EXPERIMENTAL SETUP	23
	6.3 RESULT ANALYSIS	25
	6.4 COMPARISON WITH EXISTING APPROACHES	28
	6.5 DISCUSSION	29
7	**CONCLUSION AND FUTURE WORK**	
	7.1 CONCLUSION	30
	7.2 FUTURE WORK 4.5 DISCUSSION	31
8	**REFERENCES**	32

CHAPTER 1

INTRODUCTION AND OBJECTIVES

1.1 Introduction

The rapid growth of automation has increased the need for intelligent path-planning systems capable of coordinating multiple autonomous agents in a shared environment. Applications such as automated warehouses, autonomous mobile robots, drone navigation, airport traffic management, and industrial automation require multiple agents to move simultaneously while avoiding collisions. This problem is known as **Multi-Agent Path Finding (MAPF)**.

In a MAPF problem, each agent is assigned a unique start location and a destination. The objective is to compute a collision-free path for every agent while minimizing the overall travel cost and completion time. As the number of agents increases, interactions among them become more frequent, making conflict resolution a major challenge. Therefore, developing efficient path-planning algorithms has become an important research area in Artificial Intelligence and Robotics.

Several algorithms have been proposed to solve MAPF problems. **Conflict-Based Search (CBS)** is one of the most widely used optimal algorithms because it guarantees collision-free solutions. However, its computational cost increases rapidly when many conflicts occur. **Lazy Constraints Addition Search for MAPF (LaCAM)** is designed to improve scalability by generating paths quickly, but it may experience stagnation in highly congested environments.

To address these limitations, this internship presents a Hybrid Multi-Agent Path Finding Algorithm that combines the strengths of LaCAM, Conflict-Based Search, and Machine Learning-based conflict prioritization. Instead of resolving conflicts in a fixed order, uses a Random Forest regression model to estimate the importance of each detected conflict and resolves the most critical conflict first. This approach improves the efficiency of conflict resolution while maintaining reliable collision-free path planning through the CBS fallback mechanism.

The Algorithm was implemented using Python and Pygame with an interactive graphical interface that enables users to create different environments, place agents and obstacles, execute planning algorithms, and analyse their performance. The proposed system demonstrates how machine learning can effectively support classical search algorithms for solving complex Multi-Agent Path Finding problems.

1.2 Objectives of the Internship

The main objective of this internship was to design and develop a **hybrid Multi-Agent Path Finding (MAPF) algorithm** that improves conflict resolution using Machine Learning while ensuring efficient and collision-free path planning.

The specific objectives are:

- To study the fundamentals of Multi-Agent Path Finding (MAPF) and its real-world applications.
- To understand the working principles of the Conflict-Based Search (CBS) and LaCAM algorithms.
- To implement the LaCAM algorithm for fast initial path generation.
- To detect vertex and edge conflicts occurring during multi-agent path planning.
- To extract relevant conflict features, including conflict type, conflict time step, nearby agents, nearby obstacles, makespan, and Sum of Costs (SoC).
- To train a Random Forest Regression model for predicting conflict priority.
- To integrate the Random Forest model with the Conflict-Based Search (CBS) algorithm for intelligent conflict selection and resolution.
- To implement a CBS fallback mechanism to ensure reliable collision-free path generation in complex scenarios.
- To evaluate the performance of the proposed hybrid algorithm using metrics such as execution time, makespan, Sum of Costs (SoC), number of conflicts resolved, and success rate under different numbers of agents and obstacle densities.
- To compare the performance of the proposed hybrid algorithm with existing MAPF approaches, namely CBS and LaCAM, and demonstrate its effectiveness.

1.3 Scope of the Internship

The scope of this internship is limited to the development and evaluation of a simulation-based Multi-Agent Path Finding system. The project focuses on integrating LaCAM, Conflict-Based Search, and Machine Learning into a single framework to improve conflict selection during path planning.

The developed simulator allows users to create different grid environments, add obstacles, place multiple agents, and observe their movement towards assigned goals.

The system detects conflicts during execution, predicts conflict priority using a Random Forest model, and resolves conflicts through CBS whenever necessary. Performance metrics such as execution time, makespan, sum of costs, number of conflicts, and fallback behaviour are analysed to evaluate the effectiveness of the proposed approach.

Although the project is implemented in a simulated environment, the methodology can be extended to practical applications such as warehouse robots, autonomous delivery systems, intelligent transportation, and multi-robot coordination.

CHAPTER 2

INSTITUTE PROFILE

ADDRESS

National Institute of Technology Tiruchirappalli (NIT Trichy)
Tanjore Main Road,
National Highway 83,
Tiruchirappalli – 620015,
Tamil Nadu, India.
Phone: +91-431-2503000
Website: https://www.nitt.edu

INSTITUTE PROFILE

National Institute of Technology Tiruchirappalli (NIT Trichy) was established in 1964 as the Regional Engineering College (REC), Tiruchirappalli, and was later upgraded to a National Institute of Technology in 2003. It is recognized as an Institution of National Importance by the Government of India. NIT Trichy is one of India's premier technical institutions, offering undergraduate, postgraduate, and doctoral programmes in Engineering, Science, Architecture, Management, and Humanities. The institute is known for its academic excellence, advanced research facilities, innovation, and strong industry collaborations

The institute provides state-of-the-art laboratories, research centres, modern infrastructure, and a highly qualified faculty to promote quality education and research. Through collaborations with industries and research organizations, NIT Trichy encourages students to develop innovative solutions to real-world engineering problems.

The Department of Computer Science and Engineering actively conducts research in Artificial Intelligence, Machine Learning, Data Science, Computer Networks, Cyber Security, and Multi-Agent Systems. During the internship, students gain practical exposure to research methodologies, software development, and advanced computing technologies.

VISION

To be a university globally trusted for technical excellence where learning and research integrate to sustain society and industry.

MISSION

1. To provide quality technical education through excellence in teaching, learning, and research.
2. To promote innovation and interdisciplinary research for solving real-world problems.
3. To collaborate with industries and academic institutions for technological advancement.
4. To develop skilled professionals with ethical values, leadership qualities, and social responsibility.
5. To contribute to sustainable development through scientific and technological innovation.

CHAPTER 3

DAILY WORK REPORT

DAY 1:
Reported to the organization, completed the induction process, and received an overview of the internship objectives. Discussed the basics of Multi-Agent Path Finding (MAPF) and the project roadmap.

DAY 2:
Studied the fundamentals of MAPF, including grid environments, agents, obstacles, start and goal positions, and different path planning techniques.

DAY 3:
Learned the Conflict-Based Search (CBS) algorithm, including constraint trees, conflict detection, and conflict resolution strategies.

DAY 4:
Explored the LaCAM algorithm and compared its performance with CBS. Understood its advantages in generating faster initial paths.

DAY 5:
Installed and configured the required software tools such as Python, Visual Studio Code, Pygame, Matplotlib, NumPy, and other necessary libraries.

DAY 6:
Designed the grid environment and implemented agent creation, obstacle placement, and start-goal assignment using Python.+-

DAY 7:
Implemented the LaCAM algorithm for generating initial paths and verified the generated paths using different test scenarios.

DAY 8:
Developed the conflict detection module to identify vertex conflicts and edge conflicts between multiple agents.

DAY 9:
Extracted conflict-related features such as conflict type, timestep, nearby agents, nearby obstacles, makespan, and Sum of Costs (SoC).

DAY 10:
Collected training data from multiple simulation runs and prepared the dataset for Machine Learning model training.

DAY 11:
Trained the Random Forest Regression model using the collected dataset and evaluated its prediction performance.

DAY 12:
Integrated the Machine Learning model with the CBS framework to prioritize conflicts based on predicted importance.

DAY 13:
Implemented selective replanning where only the affected agent is replanned after conflict prioritization.

DAY 14:
Added the automatic CBS fallback mechanism to ensure reliable conflict resolution when Machine Learning predictions were not effective.

DAY 15:
Developed the graphical simulation using Pygame to visualize agents, obstacles, paths, and conflict resolution.

DAY 16:
Tested the complete hybrid system on different grid sizes with varying numbers of agents and obstacles.

DAY 17:
Recorded experimental results including runtime, makespan, success rate, path cost, and number of conflicts resolved.

DAY 18:
Compared the proposed Machine Learning-guided CBS framework with traditional CBS and LaCAM algorithms using experimental observations.

DAY 19:
Prepared graphs, tables, screenshots, and analyzed the overall performance improvements of the proposed approach.

DAY 20:
Compiled all implementation details, methodology, results, conclusions, and references for the internship report.

DAY 21:
Presented the completed project to the mentor, received feedback, made final corrections, and completed the internship successfully.

CHAPTER 4

LITERATURE REVIEW / SYSTEM STUDY

4.1 Literature Review

Multi-Agent Path Finding (MAPF) has attracted significant attention because of its applications in warehouse automation, robotics, autonomous vehicles, and intelligent transportation systems. The main objective of MAPF is to generate collision-free paths for multiple agents while minimizing the overall travel cost. Over the years, several algorithms have been proposed to solve this problem, each with its own advantages and limitations.

4.1.1 Conflict-Based Search (CBS)

Conflict-Based Search (CBS) is one of the most widely used algorithms for solving MAPF problems optimally. It operates using a two-level search process. At the lower level, an A* search algorithm generates the shortest path for each individual agent. At the higher level, CBS detects conflicts between agents and resolves them by adding constraints and replanning only the affected agent.

CBS guarantees complete and optimal solutions whenever a solution exists. Since only conflicting agents are replanned, it is generally more efficient than searching all agents together. However, as the number of agents increases, the number of conflicts also increases, leading to the generation of a large Constraint Tree. Consequently, the execution time and memory consumption increase significantly, especially in highly congested environments.

4.1.2 Lazy Constraints Addition Search for MAPF (LaCAM)

LaCAM is a search-based MAPF algorithm developed to improve scalability and execution speed. Instead of constructing a large constraint tree, LaCAM generates feasible paths using lazy constraint addition and Priority Inheritance with Backtracking (PIBT). This enables the algorithm to produce solutions much faster than CBS in many scenarios.

The major advantage of LaCAM is its ability to handle large numbers of agents efficiently. However, in narrow corridors or highly congested environments, agents may repeatedly block each other, resulting in stagnation and increased execution time. Therefore, although LaCAM performs well in large environments, it may not always provide reliable performance in difficult scenarios.

4.1.3 Machine Learning-Based Conflict Prioritization

Recent research has explored the integration of Machine Learning with traditional MAPF algorithms to improve search efficiency. Instead of resolving conflicts using fixed rules, learning-based methods analyse the characteristics of each conflict and predict which conflict should be resolved first.

In this project, a Random Forest regression model is used for conflict prioritization. The model considers several conflict-related features, including conflict type, conflict timestep, nearby obstacles,nearby agents, makespan, and total path cost. Based on these features, it predicts the priority of each conflict, allowing the solver to focus on more critical conflicts before resolving less significant ones.

4.2 Problem Statement

Although CBS guarantees optimal solutions, its performance decreases when the number of conflicts increases because the Constraint Tree becomes larger. On the other hand, LaCAM generates paths quickly but may experience stagnation in highly congested environments. In addition, traditional CBS resolves conflicts in a fixed order without considering the importance of individual conflicts.

These limitations reduce the overall efficiency of Multi-Agent Path Finding, particularly in environments containing many agents and obstacles.
Therefore, there is a need for a hybrid approach that combines the speed of LaCAM, the reliability of CBS, and the intelligent decision-making capability of Machine Learning to improve conflict selection while maintaining collision-free path planning.

4.3 Proposed Methodology

To overcome the limitations of existing methods, this internship proposes a Hybrid Multi-Agent Path Finding Simulator integrating LaCAM, Conflict-Based Search, and Random Forest-based conflict prioritization.

Initially, LaCAM generates paths for all agents independently. The generated paths are analysed to identify vertex and edge conflicts. Each detected conflict is converted into a feature vector containing information about the conflict location, conflict time, nearby obstacles, nearby agents, makespan, and total path cost.

These features are provided to a trained Random Forest regression model, which predicts the priority of every conflict. Instead of resolving conflicts in their order of detection, the highest-priority conflict is selected first. The selected conflict is then resolved using the CBS constraint mechanism and A* replanning.

Whenever repeated conflicts indicate stagnation or the search exceeds the predefined limit, the system automatically switches to complete CBS search. This hybrid strategy combines the fast path generation capability of LaCAM, the optimal conflict resolution of CBS, and the adaptive conflict selection provided by Machine Learning, resulting in a more efficient and reliable MAPF solution.

Workflow of the Proposed Hybrid CBS–LaCAM Framework with Machine Learning-Guided Conflict Prioritization

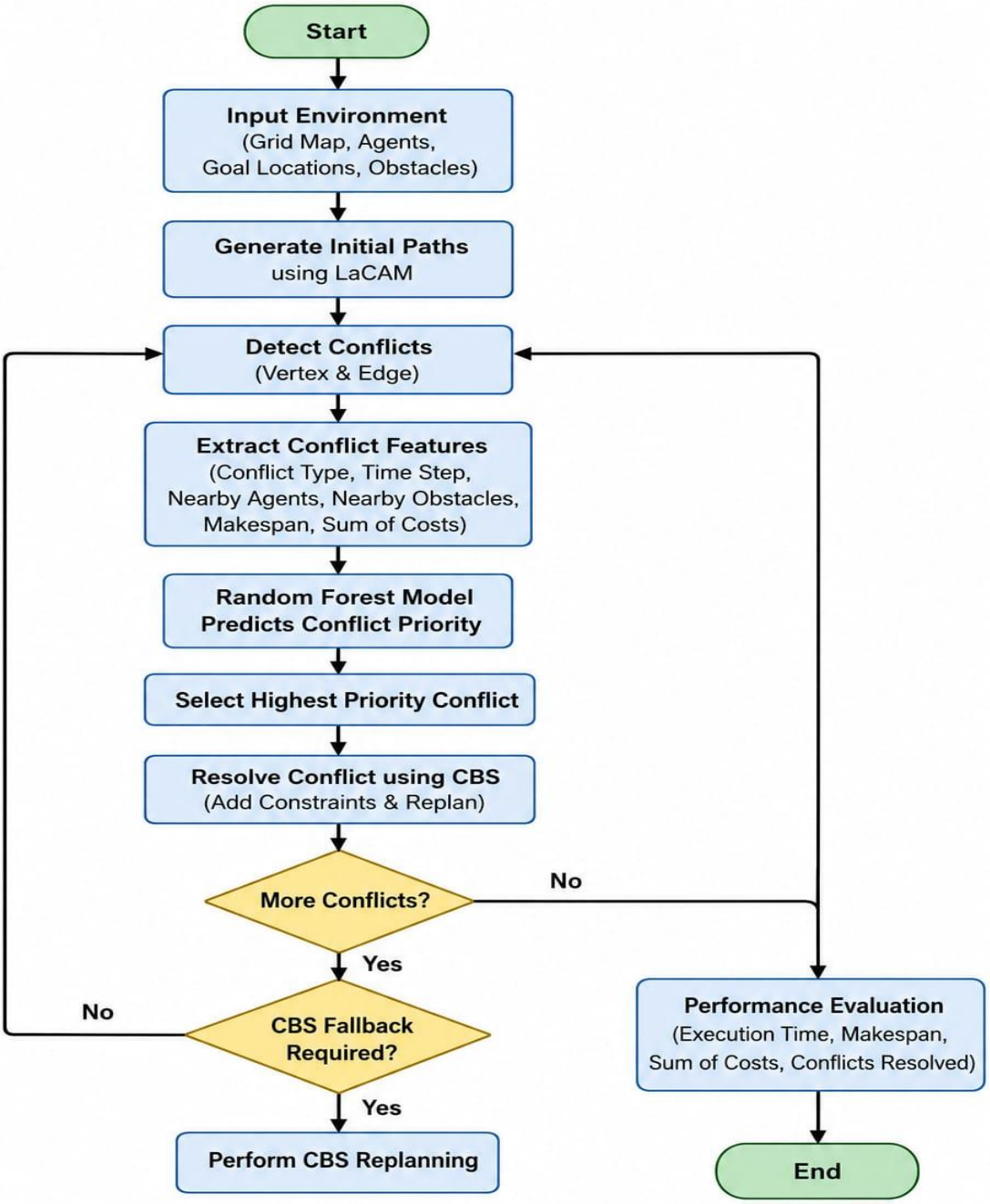


Figure 4.1

Figure 4.1 illustrates the workflow of the proposed hybrid Multi-Agent Path Finding (MAPF) framework. Initially, the user creates the simulation environment by defining the grid map, placing agents, specifying their goal locations, and adding obstacles.

The proposed workflow describes a **Machine Learning-Guided Hybrid CBS-LaCAM framework** for solving the Multi-Agent Path Finding (MAPF) problem efficiently. The framework integrates the fast path generation capability of **LaCAM**, the optimal conflict resolution mechanism of **Conflict-Based Search (CBS)**, and a **Random Forest machine learning model** for intelligent conflict prioritization. This combination aims to reduce execution time, minimize unnecessary replanning, and improve the overall efficiency of multi-agent navigation.

The workflow begins with the **input environment**, which includes the grid map, agent start positions, goal locations, and obstacles. These inputs define the environment in which multiple agents must navigate safely from their respective start locations to their destinations.

Initially, **LaCAM** generates feasible paths for all agents. LaCAM is chosen because it produces initial paths quickly, making it suitable for environments with a large number of agents. However, since these paths are generated independently, conflicts may occur during execution.

The generated paths are then analyzed during the **conflict detection** stage, where both **vertex conflicts** (two agents occupying the same grid cell at the same time) and **edge conflicts** (two agents swapping positions simultaneously) are identified. After detecting conflicts, the system extracts several important features, including conflict type, time step, nearby agents, nearby obstacles, makespan, and Sum of Costs (SOC). These features provide valuable information about the complexity and impact of each conflict.

The extracted features are provided as input to a **Random Forest model**, which predicts the priority of each conflict based on patterns learned from previously collected training data. Instead of resolving conflicts in the order they are detected, the framework selects the **highest-priority conflict**, allowing the algorithm to focus on the conflicts that have the greatest impact on the overall solution.

The selected conflict is resolved using **Conflict-Based Search (CBS)**. CBS introduces appropriate constraints for the conflicting agents and replans only the affected paths while keeping the remaining paths unchanged. This targeted replanning reduces computational overhead and improves solution quality.

After conflict resolution, the framework checks whether additional conflicts remain. If no conflicts are found, the algorithm proceeds to the **performance evaluation** stage.

Otherwise, the system determines whether **CBS fallback replanning** is required. When necessary, CBS performs additional replanning with updated constraints until a completely collision-free solution is obtained.

Finally, the proposed framework is evaluated using key performance metrics such as **execution time, makespan, Sum of Costs (SOC), and the number of conflicts resolved**. These metrics are compared with existing approaches to demonstrate the effectiveness of the proposed Machine Learning-Guided Hybrid CBS-LaCAM framework.

By combining fast initial path generation, intelligent conflict prioritization, and efficient conflict resolution, the proposed approach achieves improved scalability, reduced planning time, and better overall performance in multi-agent path planning scenarios.

CHAPTER 5

SYSTEM REQUIREMENTS AND ARCHITECTURE

5.1 System Architecture

The proposed framework follows a modular architecture in which each module performs a specific task in the Multi-Agent Path Finding (MAPF) process.

This modular design improves code maintainability, simplifies testing, and allows future enhancements without affecting the overall system.

Initially, the user creates the simulation environment by defining the grid map, placing multiple agents, assigning their goal locations, and adding obstacles.

The LaCAM module generates initial paths quickly for all agents. These paths are analyzed by the Conflict Detection module to identify both vertex and edge conflicts.

Whenever conflicts are detected, the Feature Extraction module collects important conflict characteristics such as conflict type, conflict time step, nearby agents, nearby obstacles, makespan, and Sum of Costs (SoC).

These features are provided to the trained Random Forest regression model, which predicts the priority of each detected conflict.

Based on the predicted priority, the Conflict-Based Search (CBS) module selects and resolves the most critical conflict by generating appropriate constraints and replanning the affected agent using the A* search algorithm.

If repeated conflicts indicate stagnation, the framework automatically activates the CBS fallback mechanism to ensure a valid collision-free solution.

Finally, the Performance Analysis module evaluates the effectiveness of the proposed framework by measuring execution time, makespan, Sum of Costs, conflicts resolved, planning rounds, and success rate.

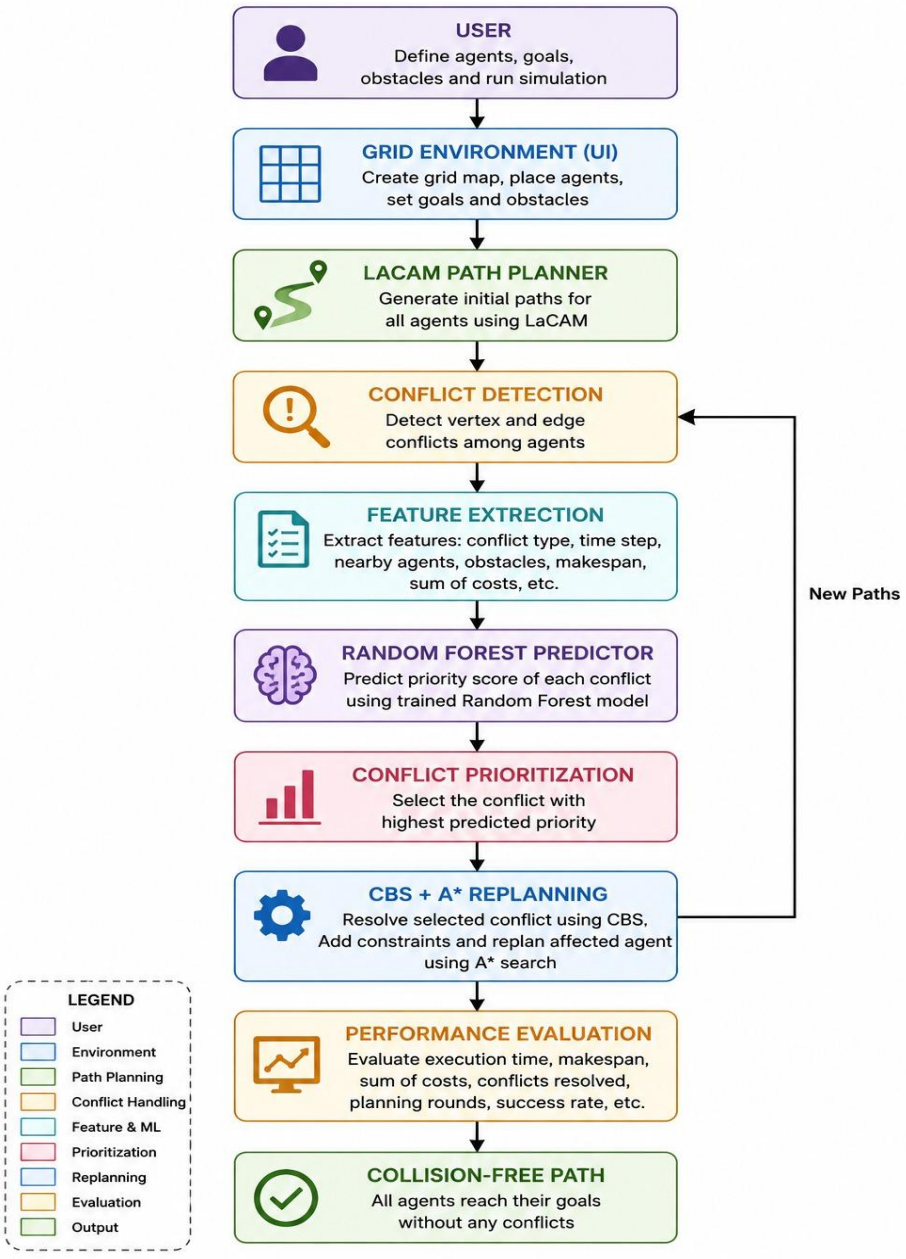


Figure 5.1

Figure 5.1: System Architecture of the Proposed Framework

5.2 Module Description

The Algorithm consists of the following modules.

5.2.1 Grid Management Module

This module creates the simulation environment. It allows users to generate grid maps, place obstacles, assign start positions, and specify goal locations for multiple agents.

5.2.2 LaCAM Path Planning Module

This module generates the initial paths for all agents using the LaCAM algorithm. It provides fast path generation without considering global conflicts, thereby reducing the initial planning time.

5.2.3 Conflict Detection Module

After the initial paths are generated, this module detects all vertex conflicts and edge conflicts among the agents. The detected conflicts are forwarded to the feature extraction module.

5.2.4 Feature Extraction Module

This module extracts important characteristics of each detected conflict, including conflict type, conflict timestep, nearby obstacles, nearby agents, makespan, and total path cost. These features serve as input to the Random Forest model.

5.2.5 Machine Learning Module

The Machine Learning module loads the trained Random Forest regression model and predicts the priority of every detected conflict. The conflict with the highest predicted priority is selected for resolution.

5.2.6 CBS Conflict Resolution Module

The CBS module resolves the selected conflict by generating appropriate constraints and replanning the affected agent using A* search. If additional conflicts are detected, the process continues until a conflict-free solution is obtained.

5.2.7 Performance Analysis Module

This module records important performance parameters including execution time, makespan, sum of costs, number of conflicts resolved, number of planning rounds, and CBS fallback information. These values are displayed through the simulator interface for analysis and comparison.

5.3 Machine Learning Model

The proposed framework employs a **Random Forest Regression** model to improve conflict prioritization during Multi-Agent Path Finding. Instead of resolving conflicts in a fixed order, the model predicts the importance of each detected conflict based on several extracted features. This intelligent prioritization reduces unnecessary replanning operations and improves the efficiency of the hybrid CBS–LaCAM framework.

The Random Forest model was trained using conflict data collected from multiple simulation environments. An 80:20 train-test split was used to evaluate the model. After training, the model was saved and integrated into the simulator for real-time conflict priority prediction.

Table 5.1 Machine Learning Parameters

Parameter	Value
Algorithm	Random Forest Regressor
Number of Trees	200
Train-Test Split	80-20
Random State	42
Target Variable	Conflict Priority

5.4 Conflict Feature Extraction

During simulation, every detected conflict is converted into a feature vector before being passed to the Random Forest model. These features describe the characteristics of the conflict and help the model determine which conflict should be resolved first.

Table 5.2 Extracted Conflict Features

Feature	Description
Conflict Type	Vertex or Edge conflict
Conflict Time Step	Time at which the conflict occurs
Nearby Agents	Number of neighboring agents
Nearby Obstacles	Number of nearby obstacles
Makespan	Total completion time
Sum of Costs(SoC)	Total path cost of all agents

CHAPTER 6

DEVELOPMENT AND RESULT ANALYSIS

6.1 System Development and Implementation

The Algorithm was developed using Python with Pygame as the graphical interface. The implementation was carried out in a modular manner so that each component could be developed, tested, and integrated independently.

Figure 6.1: Initial Grid Environment of the MAPF Simulator

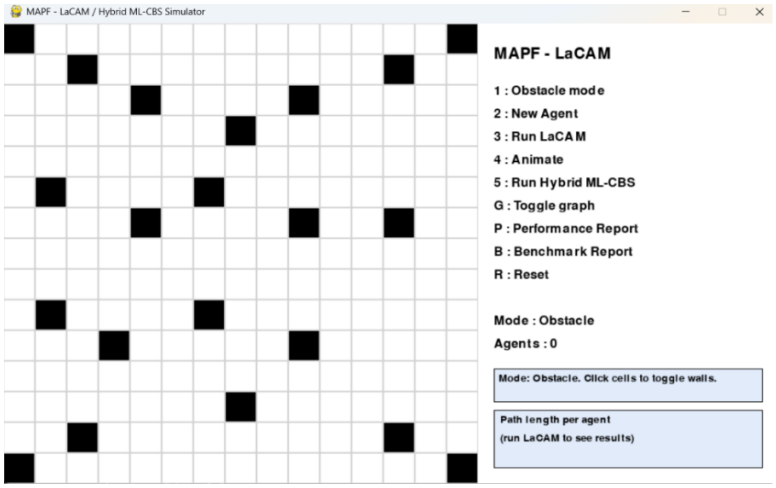


Figure 6.1 shows the initial interface of the proposed simulator. The user creates the grid environment by placing obstacles before adding agents and goal locations. This environment serves as the input for the path planning process.

Initially, the simulator was designed to allow users to create different grid environments by placing obstacles, agents, and their corresponding goal locations. Each agent is assigned a unique colour to make its movement easy to observe during simulation.

Figure 6.2: Grid Environment with Multiple Agents and Goal Locations

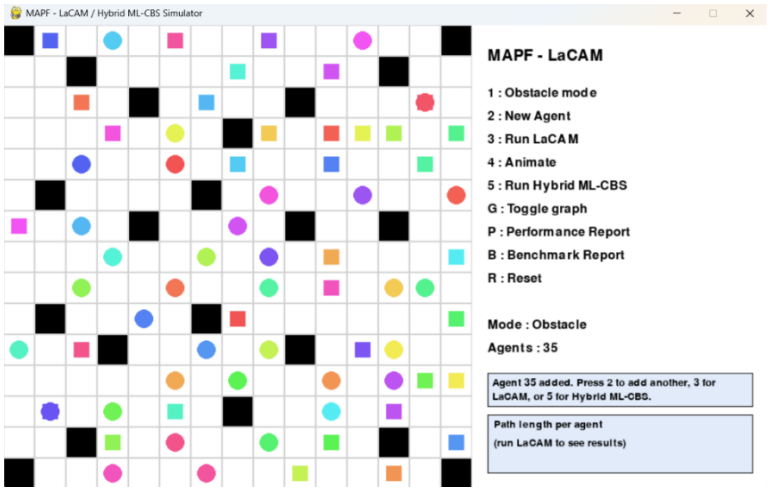


Figure 6.2 illustrates the grid after multiple agents and their corresponding goal locations have been added. Each agent is represented using a unique colour to distinguish its movement during simulation.

After creating the environment, LaCAM is executed to generate the initial path for every agent. Since LaCAM focuses on fast path generation, the initial paths may contain vertex or edge conflicts. Therefore, a conflict detection module continuously analyses the generated paths to identify all possible conflicts.

Figure 6.3: Initial Path Generation Using LaCAM

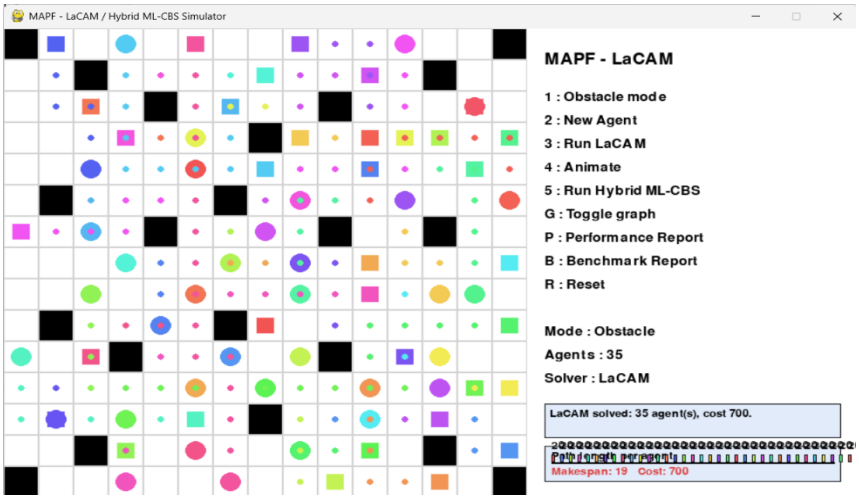


Figure 6.3 presents the initial paths generated by the LaCAM algorithm. The generated paths provide feasible routes for all agents before conflict resolution is performed.

For every detected conflict, important features such as conflict type, conflict timestep, nearby obstacles, nearby agents, makespan, and sum of costs are extracted automatically. These features are supplied to the trained Random Forest regression model, which predicts the priority of each conflict.

Figure 6.4: Agent Movement During Simulation

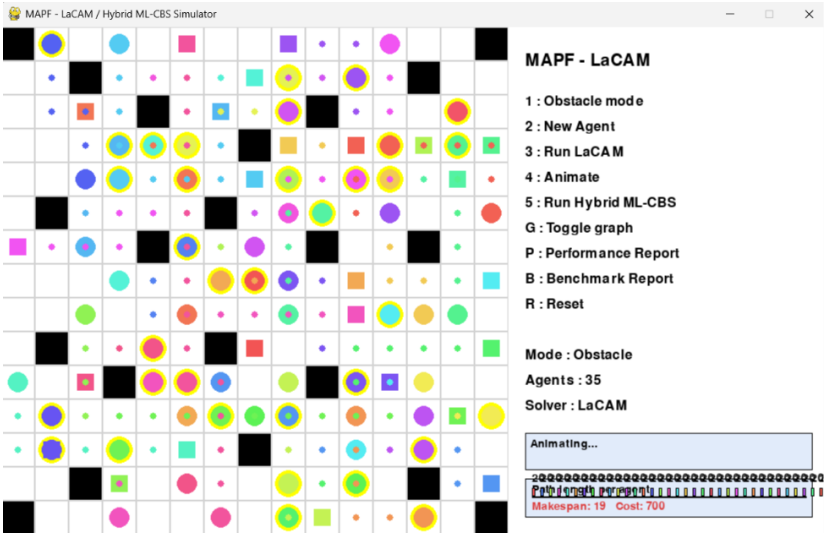


Figure 6.4 shows the animation of agents moving along their planned paths. During execution, the simulator continuously monitors agent movements to identify potential conflicts.

Instead of resolving conflicts sequentially, selects the conflict with the highest predicted priority.

The selected conflict is then resolved using the Conflict-Based Search algorithm by introducing suitable constraints and replanning only the affected agent. This process continues until all conflicts are eliminated.

Figure 6.5: Collision-Free Path after Path Planning

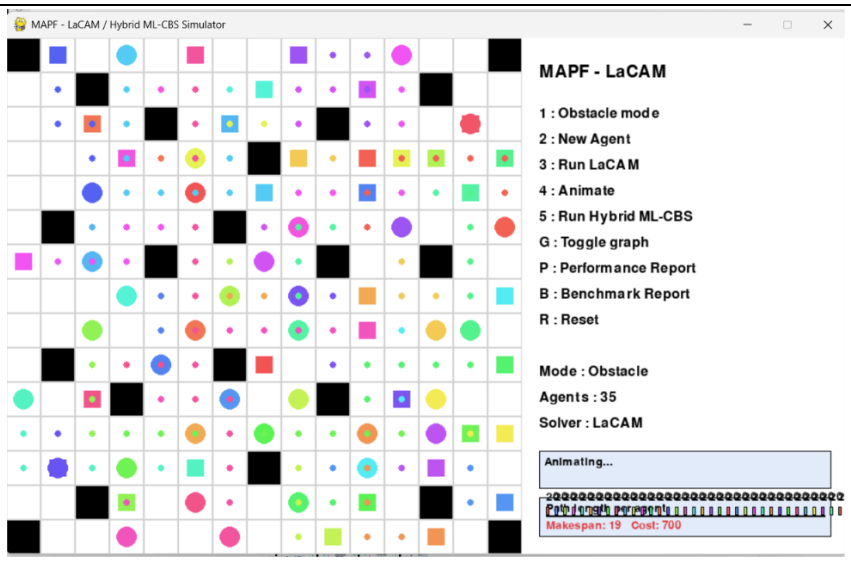


Figure 6.5 displays the final planned paths after successful path planning. The agents reach their assigned destinations while avoiding collisions.

To improve system reliability, an automatic CBS fallback mechanism is included. If repeated conflicts indicate stagnation or the learning-guided conflict resolution exceeds the predefined limit, the system switches to complete CBS search. This guarantees that a collision-free solution is produced whenever one exists.

The Algorithm also provides a performance analysis panel that displays important execution statistics, allowing users to compare different planning scenarios easily.

6.2 Experimental Setup

The performance of was evaluated using multiple grid environments with different obstacle layouts and varying numbers of agents. These experimental scenarios were selected to analyse the behaviour of the proposed hybrid algorithm under simple, moderate, and highly congested environments. Testing the algorithm under different conditions helps determine its efficiency, scalability, reliability, and ability to resolve conflicts as the complexity of the environment increases.

The Random Forest regression model was trained using conflict data generated from several simulation environments. The extracted conflict features were used to learn the importance of each conflict during path planning. After training, the model was integrated into the simulator to perform real-time conflict prioritization during execution.

To evaluate the overall performance of Conflict several performance parameters were selected. Each parameter measures a different aspect of the proposed system.

Execution Time

Execution time represents the total time taken by to generate collision-free paths for all agents. This parameter was selected because many real-world applications, such as warehouse automation and autonomous robots, require fast path planning. Lower execution time indicates better computational efficiency.

Number of Conflicts Detected

This parameter indicates the total number of vertex and edge conflicts identified after the initial paths were generated. It helps measure the complexity of the environment. As the number of agents or obstacles increases, the number of detected conflicts also increases.

Number of Conflicts Resolved

This parameter represents the number of conflicts successfully eliminated by the proposed algorithm. It was chosen to evaluate the effectiveness of the Machine Learning-guided conflict prioritization strategy. Successfully resolving conflicts without introducing new collisions indicates good planning performance.

Makespan

Makespan is defined as the total time required for the last agent to reach its destination. This parameter evaluates the overall completion time of the multi-agent system. A lower makespan indicates better coordination among agents and improved scheduling efficiency.

Sum of Costs (SoC)

The Sum of Costs is the total path length travelled by all agents. This parameter measures the overall quality of the generated solution. Lower SoC values indicate that agents follow shorter and more efficient paths while avoiding unnecessary movement.

Number of Planning Rounds

Planning rounds refer to the number of iterations required to remove all conflicts before obtaining a collision-free solution. This parameter was included because an efficient conflict prioritization strategy should reduce the number of replanning iterations.

CBS Fallback Behaviour

It combines Machine Learning with Conflict-Based Search. However, in highly congested environments, the learning-guided strategy may become ineffective. Therefore, the CBS fallback behaviour was analysed to evaluate the robustness of the hybrid framework. A successful fallback mechanism ensures that valid solutions can still be obtained in difficult scenarios.

Success Rate

The success rate represents the percentage of test cases in which the simulator successfully generated collision-free paths. This parameter was selected to evaluate the reliability and correctness of the proposed hybrid algorithm across different MAPF environments.

The combination of these evaluation parameters provides a comprehensive assessment of by measuring computational efficiency, solution quality, conflict management capability, and overall system reliability.

6.3 Result Analysis

The experimental results demonstrate that successfully generated collision-free paths in most of the tested environments. In scenarios with fewer agents and lower obstacle density, conflicts

were resolved quickly using the Machine Learning-guided conflict prioritization without requiring CBS fallback.

Figure 6.6: Performance Evaluation of the Proposed Hybrid ML–CBS Framework

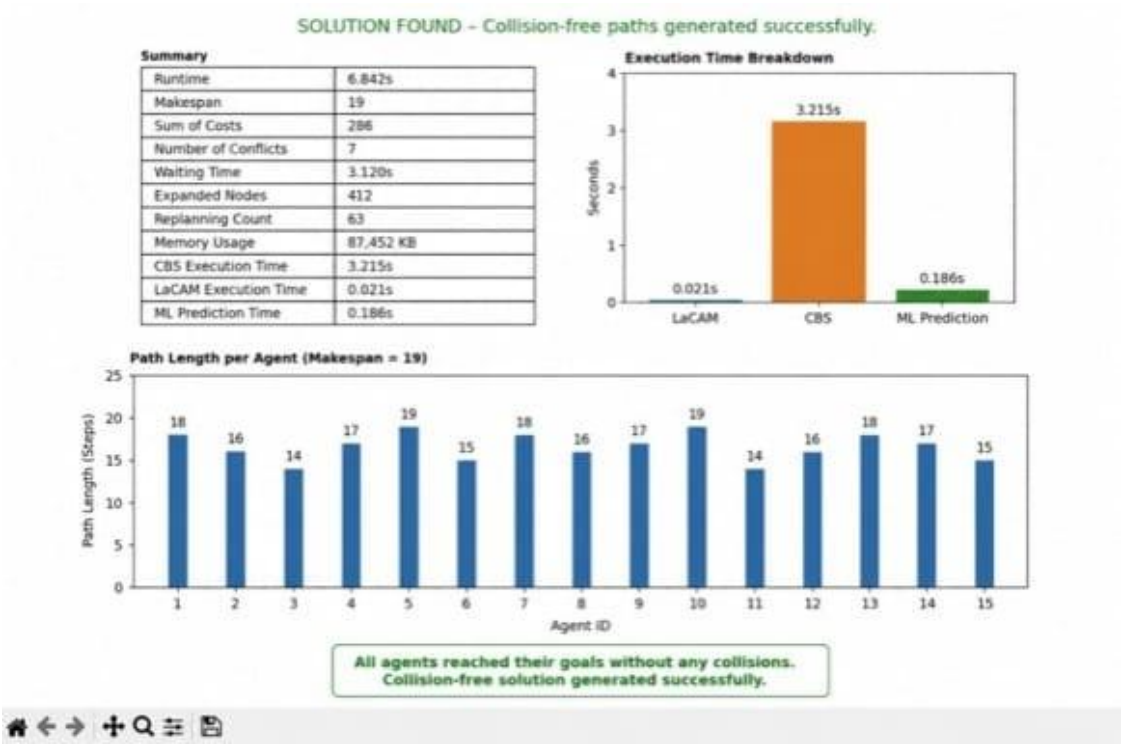


Figure 6.6 presents the performance evaluation of the proposed Machine Learning-Guided Hybrid CBS-LaCAM framework. It reports key performance metrics, including execution time, conflict statistics, planning rounds, memory usage, and the execution times of LaCAM, CBS, and the Machine Learning prediction module.

As the number of agents and obstacles increased, the complexity of the MAPF problem also increased because more conflicts occurred during path planning. Even under these challenging conditions, the Random Forest model effectively identified the most important conflicts by analysing local congestion around the conflict location. Resolving these high-priority conflicts first reduced unnecessary replanning operations and improved the overall efficiency of the search.

Figure 4.7: Benchmark Performance Comparison

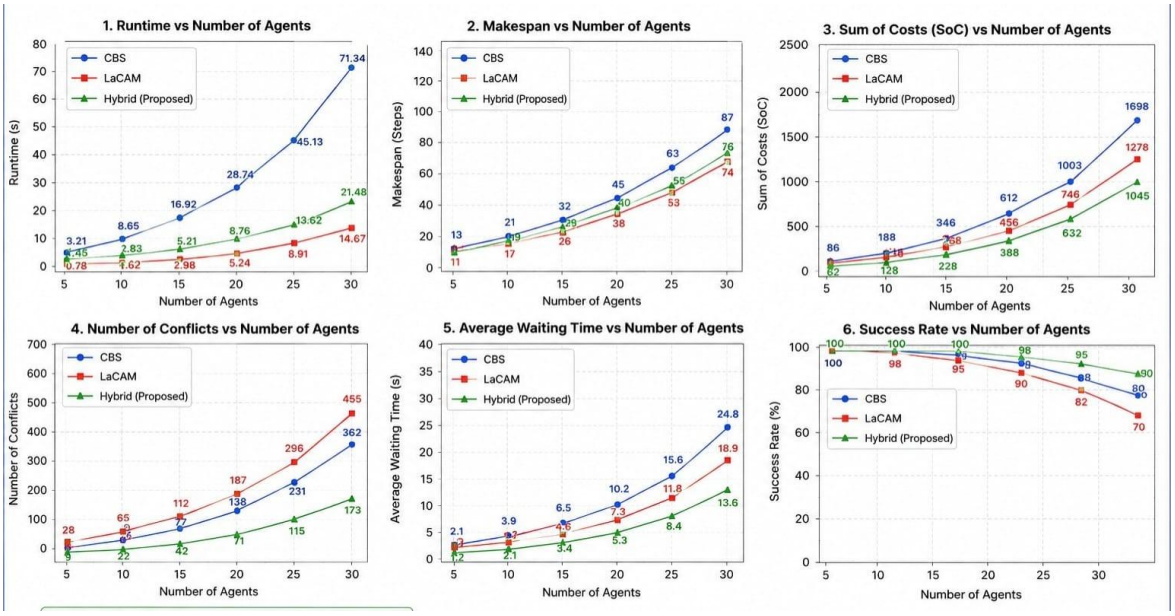


Figure 6.7 compares the performance of CBS, LaCAM, and the proposed Machine Learning-Guided Hybrid CBS-LaCAM framework under different numbers of agents. The benchmark includes execution time, makespan, Sum of Costs (SoC), number of conflicts, and average waiting time, demonstrating the improved scalability and efficiency of the proposed approach.

The feature importance analysis showed that **nearby agents** and **nearby obstacles** contributed most significantly to conflict priority prediction.

This observation indicates that local congestion has a greater impact on conflict severity than the identity or location of individual agents.

Consequently, the Machine Learning model was able to identify conflicts that were more likely to delay the planning process. In highly congested environments, repeated conflicts occasionally caused the learning-guided strategy to stagnate. Instead of continuing ineffective conflict resolution, automatically activated the CBS fallback mechanism.

This ensured that the simulator continued searching for a valid collision-free solution while maintaining the correctness of the planning process. Overall, the experimental results demonstrate that integrating Machine Learning with CBS improves conflict selection, reduces unnecessary replanning in many scenarios, and provides reliable performance across different MAPF environments.

6.4 Comparison with Existing Approaches

To evaluate the effectiveness of its performance was compared with traditional Conflict-Based Search (CBS) and LaCAM.

Parameter	**CBS**	**LaCAM**	**Proposed ML-Guided Hybrid CBS-LaCAM**
Initial Path Generation	Slow	Fast	Fast
Conflict Selection	Constraint-based	Configuration-based	ML-guided Conflict Prioritization
Conflict Resolution	CBS-Based	Local Search-Based	CBS with ML-Guided Prioritization
Performance in Congested Maps	Execution time increases significantly	May stagnate dense environments	Better conflict handling with intelligent prioritization and CBS fallback
Reliability	High	Moderate	High
Machine Learning Support	No	No	Yes(Random Forest)
Collision-Free Guarantee	Yes	Yes	Yes
Scalability	Moderate	High	High
Execution Efficiency	Moderate	High	High

The comparison shows that combines the strengths of both CBS and LaCAM. It preserves the reliability and correctness of CBS while benefiting from the fast initial path generation of LaCAM.

In addition, the Random Forest model improves conflict selection by prioritizing more critical conflicts instead of following a fixed ordering strategy. This reduces unnecessary replanning in many scenarios and improves the overall efficiency of the hybrid framework.

6.5 Discussion

The proposed framework enhances CBS by using a Random Forest model to intelligently prioritize conflicts, reducing search effort and improving planning efficiency without modifying the core algorithm.

The CBS fallback mechanism ensures reliable conflict resolution. In the future, the framework can be extended using Deep Learning or Reinforcement Learning and evaluated on larger-scale benchmark datasets to improve scalability and real-world performance.

CHAPTER 7

CONCLUSION AND FUTURE WORK

7.1 Conclusion

This project presented a **Machine Learning-Guided Conflict Prioritization for Hybrid CBS-LaCAM Path Planning** framework to improve the efficiency of Multi-Agent Path Finding (MAPF). The proposed approach combines the fast initial path generation capability of **LaCAM**, the optimal conflict resolution strategy of **Conflict-Based Search (CBS)**, and a **Random Forest Regression** model for intelligent conflict prioritization.

The framework first generates feasible paths using LaCAM and then detects vertex and edge conflicts. Relevant conflict features, including conflict type, conflict timestep, nearby agents, nearby obstacles, makespan, and Sum of Costs (SoC), are extracted and provided to the Random Forest model. The predicted priority scores enable the framework to resolve the most critical conflicts first, reducing unnecessary replanning operations.

The implementation was carried out using Python and Pygame, allowing interactive simulation of multiple agents in different grid environments. Experimental results demonstrated that the proposed framework effectively generates collision-free paths while maintaining good planning efficiency across different scenarios. The integration of Machine Learning with CBS improves conflict selection without changing the core search algorithm. In challenging environments, the CBS fallback mechanism ensures that the framework continues searching for a valid solution, thereby improving the robustness and reliability of the overall system.Overall, the proposed hybrid framework successfully combines the strengths of LaCAM, CBS, and Machine Learning to provide an efficient and scalable solution for Multi-Agent Path Finding problems.

7.2 Future Work

Although the proposed framework achieves effective conflict prioritization and collision-free path planning, several improvements can be explored in future work.

- Replace the Random Forest model with advanced Deep Learning models such as Graph Neural Networks (GNNs) or Reinforcement Learning to improve conflict prioritization.
- Evaluate the framework using larger benchmark datasets with hundreds or thousands of agents to analyse scalability.
- Extend the framework to dynamic environments where obstacles and goals can change during execution.
- Investigate adaptive learning techniques that continuously update the prediction model using newly generated conflict data.
- Integrate the framework with real-world robotic systems, such as automated warehouse robots, autonomous delivery robots, and smart manufacturing systems.
- Optimize memory usage and execution time for large-scale MAPF applications.
- Compare the proposed framework with additional state-of-the-art MAPF algorithms to further validate its performance.

These enhancements can improve the scalability, adaptability, and practical applicability of the proposed framework in real-world multi-agent systems.

CHAPTER 8

REFERENCES

1. J. Li, *Multi-Agent Path Finding (MAPF) Research Website*. Available: https://mapf.info/. Accessed: July 2026.

2. K. Okumura, "LaCAM: Search-Based Algorithm for Quick Multi-Agent Pathfinding," *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, vol. 37, no. 9, pp. 11331–11339, 2023.

3. G. Sharon, R. Stern, A. Felner, and N. R. Sturtevant, "Conflict-Based Search for Optimal Multi-Agent Path Finding," *Artificial Intelligence*, vol. 219, pp. 40–66, 2015.

4. S. Wang, H. Xu, Y. Zhang, J. Lin, C. Lu, X. Wang, and W. Li, "Where Paths Collide: A Comprehensive Survey of Classic and Learning-Based Multi-Agent Pathfinding," *arXiv preprint*, 2025.

5. P. Hart, N. Nilsson, and B. Raphael, "A Formal Basis for the Heuristic Determination of Minimum Cost Paths," *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.

6. L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

7. S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Pearson, 2021.

8. C. M. Bishop, *Pattern Recognition and Machine Learning*. New York, NY, USA: Springer, 2006.

9. F. Pedregosa *et al*., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

10. T. Oliphant, *Guide to NumPy*, 2nd ed. USA: CreateSpace Independent Publishing Platform, 2015.

11. J. D. Hunter, "Matplotlib: A 2D Graphics Environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007.

12. Pygame Community, *Pygame Documentation*. Available: https://www.pygame.org/docs/. Accessed: July 2026.

13. Python Software Foundation, *Python 3.11 Documentation*. Available: https://docs.python.org/3/. Accessed: July 2026.

14. F. Chollet, *Deep Learning with Python*, 2nd ed. Manning Publications, 2021.

15. M. Wooldridge, *An Introduction to MultiAgent Systems*, 2nd ed. John Wiley & Sons, 2009.